

```
; =====  
; PureBasic OOP - Manuel de Référence (Français)  
; Documentation officielle du transpileur et du framework GUI OOP  
; Author:      MicrodevWeb  
; =====
```

Manuel de Référence PureBasic OOP (Français)

Bienvenue dans la documentation officielle du transpileur et du framework Orienté Objet pour PureBasic.

1. Les Fondements de la Programmation Orientée Objet (POO)

La Programmation Orientée Objet (POO) est un paradigme de programmation qui structure une application autour de **données**.

Elle repose sur 5 piliers fondamentaux :

1.1 Les Classes et les Objets

- **La Classe** : C'est le plan de construction (ou moule) qui définit la structure (attributs / champs) et le comportement (méthodes).
- **L'Objet (ou Instance)** : C'est une occurrence concrète créée en mémoire à partir d'une classe (par exemple, la classe `Chien`).

1.2 L'Encapsulation

L'encapsulation permet de regrouper les données et les fonctions qui les manipulent, tout en protégeant les données internes contre les accès non autorisés.

- **Public** : Accessible depuis n'importe où (à l'intérieur et à l'extérieur de l'objet).
- **Protected** : Accessible uniquement par la classe qui le déclare et par ses classes filles (héritières).
- **Private** : Strictement réservé à la classe qui le déclare.

1.3 L'Héritage (`Extends`)

L'héritage permet à une classe dérivée (fille) de réutiliser et d'étendre les attributs et les méthodes d'une classe de base (parente).

1.4 Le Polymorphisme (Dispatch Dynamique via VTable)

Le polymorphisme permet de manipuler différents types d'objets dérivés à travers une référence commune vers leur classe parente.

1.5 L'Abstraction (Classes et Méthodes Abstraites)

L'abstraction permet de définir un concept général sans en fournir toute l'implémentation :

- **Classe Abstraite** (`Abstract Class`) : Une classe incomplète servant de modèle ou de contrat. Elle **ne peut pas être instanciée**.
- **Méthode Abstraite** (`Abstract Method`) : Un prototype de méthode sans corps d'implémentation. Toute classe fille concrète a à implémenter cette méthode.
- **Méthode Concrète / Par Défaut** : Une classe abstraite peut aussi contenir des méthodes avec un code par défaut, que la classe dérivée peut hériter.

2. Syntaxe & Grammaire Objet PureBasic (.pbo)

2.1 Déclaration des Classes Abstraites et Concrètes

```
`` `oop  
;  
; -----  
; 1. CLASSE ABSTRAITE (Modèle de base / Contrat)  
; -----  
Abstract Class FormeGeometrique  
    Protected nom.s  
    Protected couleur.s  
  
    ; Constructeur  
    Public Method Init(nom_p.s, couleur_p.s)  
  
    ; Méthodes Abstraites (Obligatoires dans les classes filles concrètes)
```

```
Public Abstract Method.d CalculerAire()
Public Abstract Method.d CalculerPerimetre()
Public Abstract Method Dessiner()

; Méthode concrète avec implémentation par défaut dans la classe abstraite
Public Method AfficherInfos()

; Destructeur
Public Method Free()
EndClass

; -----
; 2. CLASSE CONCRÈTE (Hérite de la classe abstraite)
; -----
Class Rectangle Extends FormeGeometrique
Protected largeur.d
Protected hauteur.d

Public Method Init(nom_p.s, couleur_p.s, l.d, h.d)

; Implémentation obligatoire des méthodes abstraites du parent
Public Method.d CalculerAire()
Public Method.d CalculerPerimetre()
Public Method Dessiner()

; Surcharge de la méthode concrète
Public Method AfficherInfos()

Public Method Free()
EndClass
```
```

---

## 2.2 Surcharge des Constructeurs (`Init`) & Règle Explicite

Dans PureBasic OOP, une classe peut définir **\*\*plusieurs constructeurs surchargés\*\*** pour s'adapter aux différents besoins (du ca

```
```oop
Class Personne
Protected nom.s
Protected age.i

; Constructeur 1: Sans paramètre
Public Method Init()
This\nom = "Anonyme"
This\age = 0
EndMethod

; Constructeur 2: Nom seul
Public Method Init(nom_p.s)
This\nom = nom_p
This\age = 0
EndMethod

; Constructeur 3: Nom et Âge
Public Method Init(nom_p.s, age_p.i)
This\nom = nom_p
This\age = age_p
EndMethod
EndClass

; Instanciation selon le constructeur choisi :
Define *p1.Personne = New Personne()
Define *p2.Personne = New Personne("Alice")
Define *p3.Personne = New Personne("Bob", 30)
```

```
...

```

> **Règle des constructeurs explicites** :

> Chaque classe définit explicitement ses propres constructeurs. Une classe fille n'hérite pas automatiquement des constructeurs

3. Framework GUI Réactif & Composants (`src/ui/`)

Le framework GUI fournit des contrôles et des panneaux de mise en page réactifs (Responsive Layouts) permettant de concevoir

3.1 Liste Complète des Constructeurs des Contrôles UI

| Composant | Constructeurs Disponibles (du plus simple au plus complet) | Description & Paramètres |

| :--- | :--- | :--- |

| **UI::Button** | `Init(text.s)`
 `Init(text.s, w.i, h.i)`
 `Init(x.i, y.i, w.i, h.i, text.s)`
 `Init(x.i, y.i, w.i, h.i, text.s, flags.i)` | Bouton

| **UI::TextBox** | `Init()`
 `Init(defaultText.s)`
 `Init(defaultText.s, w.i, h.i)`
 `Init(x.i, y.i, w.i, h.i, defaultText.s)`
 `Init(x.i, y.i, w.i, h.i, defaultText.s, flags.i)` | Champ de texte

| **UI::Label** | `Init(text.s)`
 `Init(text.s, w.i, h.i)`
 `Init(x.i, y.i, w.i, h.i, text.s)`
 `Init(x.i, y.i, w.i, h.i, text.s, flags.i)` | Libellé

| **UI::CheckBox** | `Init(text.s)`
 `Init(text.s, checked.b)`
 `Init(text.s, w.i, h.i, checked.b)`
 `Init(x.i, y.i, w.i, h.i, text.s, flags.i, checked.b)` | Case à cocher

| **UI::ComboBox** | `Init()`
 `Init(w.i, h.i)`
 `Init(x.i, y.i, w.i, h.i)`
 `Init(x.i, y.i, w.i, h.i, flags.i)` | Liste déroulante sélectionnable

| **UI::ProgressBar** | `Init()`
 `Init(min.i, max.i)`
 `Init(min.i, max.i, w.i, h.i)`
 `Init(x.i, y.i, w.i, h.i, min.i, max.i, flags.i)` | Barre de progression

| **UI::Slider** | `Init()`
 `Init(min.i, max.i)`
 `Init(min.i, max.i, w.i, h.i)`
 `Init(x.i, y.i, w.i, h.i, min.i, max.i, flags.i)` | Curseur

| **UI::ListItemIcon** | `Init(title.s, colWidth.i)`
 `Init(title.s, colWidth.i, flags.i)`
 `Init(x.i, y.i, w.i, h.i, title.s, colWidth.i)`
 `Init(x.i, y.i, w.i, h.i, title.s, colWidth.i, flags.i)` | Icône avec libellé

| **UI::Controls::ToggleSwitch** | `Init()`
 `Init(checked.b)`
 `Init(w.i, h.i, checked.b)`
 `Init(x.i, y.i, w.i, h.i, checked.b)` | Interrupteur

3.2 Panneaux de Disposition Réactive (Layouts)

| Panneau Layout | Constructeurs Disponibles | Rôle & Comportement |

| :--- | :--- | :--- |

| **UI::Layouts::StackPanel** | `Init()`
 `Init(orientation.i)`
 `Init(orientation.i, spacing.i)`
 `Init(orientation.i, spacing.i, w.i, h.i)` | Panneau de disposition en empilement

| **UI::Layouts::DockPanel** | `Init()`
 `Init(lastChildFill = #True)`
 `Init(lastChildFill.b, w.i, h.i)` | Ancre les enfants à un bord

| **UI::Layouts::Grid** | `Init()`
 `Init(w.i, h.i)` | Grille 2D flexible avec dimensionnement en pixels, "Auto" ou proportionnel

| **UI::Window** | `Init(title.s)`
 `Init(title.s, w.i, h.i)`
 `Init(title.s, w.i, h.i, flags.i)`
 `Init(title.s, x.i, y.i, w.i, h.i, flags.i, parent.i)` | Fenêtre

4. Exemple d'Application Réactive Multi-Fenêtres

```
...oop

```

```
XIncludeFile "ui/UI.pbo"

```

```
Using UI

```

```
Using UI::Layouts

```

```
Class MainWindow Extends UI::Window {

```

```
    Protected *rootDock.DockPanel

```

```
    Protected *toolbar.StackPanel

```

```
    Protected *btnNew.Button

```

```
    Protected *table.ListIcon

```

```
    Public Method Init() {

```

```
        ; Fenêtre 800x600 centrée et redimensionnable

```

```
        Super::Init("Gestionnaire de Contacts", 800, 600)

```

```
        ; Panneau racine Dock

```

```
        This\*rootDock = New DockPanel(#True)

```

```
        This\SetRootComponent(This\*rootDock)

```

```
        ; Barre d'outils en haut (Dock Top)

```

```
        This\*toolbar = New StackPanel(#UI_Orientation_Horizontal, 8)
    }
EndClass

```

```

    This\*btnNew = New Button("+ Nouveau Contact")
    This\*toolbar\AddChild(This\*btnNew)
    This\*rootDock\AddDockChild(This\*toolbar, #UI_Dock_Top)

    ; Table centrale (remplit tout l'espace restant)
    This\*table = New ListIcon("Nom", 200)
    This\*table\AddColumn(1, "Email", 250)
    This\*rootDock\AddDockChild(This\*table, #UI_Dock_Fill)
}

Define *app.UI::Application = New UI::Application()
Define *win.MainWindow = New MainWindow()
*app\Run()
```
```

---

## 5. Guide de Compilation & Exécution

Transpiler un fichier source `.pbo` vers `.pb` :

```

```cmd
"compiler/transpiler.exe" "src/main.pbo" "src/main_generated.pb"
```
```

Compiler avec le compilateur officiel PureBasic :

```

```cmd
"C:\Program Files\PureBasic\Compilers\pbcompiler.exe" "src/main_generated.pb" /EXE "src/app.exe" /THREAD /UNICODE /XP /U
```
```